

Video 1: RPKM, FPKM and TPM, Clearly Explained!!!

Link: <https://youtu.be/Rsc5znwR5FA>

This video explains why we need to **normalize** read counts in RNA-seq (gene expression) data and details the three most common methods.

1. The Problem: Why We Can't Use Raw Read Counts

When you sequence two RNA samples (e.g., "Sample 1" and "Sample 2"), you can't just directly compare the raw number of reads that mapped to a gene. There are two major biases:

1. **Sequencing Depth Bias:** You might have simply sequenced Sample 1 *deeper* than Sample 2.
 - *Example:* Sample 1 has 10 million total reads, and Sample 2 has 20 million.
 - A gene in Sample 2 with 200 reads might look twice as expressed as a gene in Sample 1 with 100 reads, but this is just because the *entire library* for Sample 2 was sequenced twice as much.
2. **Gene Length Bias:** Longer genes will naturally get more reads than shorter genes, even if they are expressed at the *same level* (i.e., the same number of RNA transcripts are present in the cell).
 - *Example:* Gene A (10,000 base pairs) will have 10x more reads than Gene B (1,000 base pairs), assuming they are both being transcribed at the same rate.

The Goal of Normalization: To remove these technical biases so you can make a fair comparison of gene expression between samples and between genes.

2. The Normalization Methods

All three methods are variations of "**Reads Per Kilobase of transcript, per Million mapped reads**". The only difference is the *order of operations*.

RPKM: Reads Per Kilobase Million (Used for Single-End Sequencing)

1. **Normalize for Sequencing Depth (The "M"):** First, you get your "per-million" scaling factor. You count the total reads in your sample and divide by 1,000,000.
 - *Example:* 10,000,000 total reads / 1,000,000 = **10**.
2. **Calculate RPM (Reads Per Million):** You divide each gene's raw read count by this scaling factor.
 - *Example:* Gene A (100 reads) / 10 = **10 RPM**.

3. **Normalize for Gene Length (The "K"):** Finally, you divide the RPM by the gene's length in *kilobases* (thousands of bases).

- *Example:* Gene A (2,000 bases) is 2kb.
- $10 \text{ RPM} / 2\text{kb} = \mathbf{5 \text{ RPKM}}$.

FPKM: Fragments Per Kilobase Million (Used for Paired-End Sequencing)

- This is conceptually **identical to RPKM**.
- The only difference is that with paired-end sequencing, you count "fragments" (where two reads can map to one fragment) instead of individual "reads."
- The calculation steps and, more importantly, the *bias* are the same as RPKM.

The Problem with RPKM / FPKM: The "per-million" scaling factor (Step 1) is calculated from the *total* number of reads. This total is dominated by a few highly-expressed, *very long* genes. This makes the scaling factor unreliable and can lead to inaccurate comparisons *between* samples.

TPM: Transcripts Per Million (The Preferred Method)

TPM fixes the problem with RPKM/FPKM by simply **reversing the order of operations**.

1. **Normalize for Gene Length (The "K"):** First, you divide each gene's raw read count by its length in kilobases. This gives you **RPK (Reads Per Kilobase)**.

- *Example:* Gene A (100 reads) / 2kb = **50 RPK**.
- *Example:* Gene B (50 reads) / 1kb = **50 RPK**.

2. **Normalize for Sequencing Depth (The "M"):** Next, you add up *all* the RPK values in your sample to get a "total RPK" number. You then divide this total by 1,000,000 to get your "per-million" scaling factor.

- *Example:* Total RPK for the whole sample = 500,000.
- Scaling factor = $500,000 / 1,000,000 = \mathbf{0.5}$.

3. **Calculate TPM:** Finally, you divide each gene's RPK (from Step 1) by this new scaling factor.

- *Example:* Gene A (50 RPK) / 0.5 = **100 TPM**.
- *Example:* Gene B (50 RPK) / 0.5 = **100 TPM**.

The Benefit of TPM: By normalizing for gene length first, we see that Gene A and Gene B are actually expressed at the same level (100 TPM). The key advantage is that the **sum of all TPMs in every sample is the same (1,000,000)**. This makes it a much more stable and reliable statistic for comparing expression *between* samples.

Video 2: Principal Component Analysis (PCA) Main Ideas

Link: https://youtu.be/VX_M3tlyiYk

This video explains the main ideas behind **Principal Component Analysis (PCA)**, a powerful method for **dimension reduction**.

1. The Problem: Too Many Dimensions

- We often measure thousands of variables (or "dimensions") for each sample.
- *Example:* In an RNA-seq experiment, if you measure 20,000 genes, your data has **20,000 dimensions**.
- It is **impossible to visualize** this. You can't make a 20,000-dimension graph to see how your samples relate to each other.

2. The Solution: PCA

- PCA is a technique that **"flattens"** all of that high-dimensional data into a simple **2D plot**.
- It does this by summarizing all 20,000 variables into two new, artificial variables called **Principal Components**.
 - **Principal Component 1 (PC1):** This is the **x-axis** of the PCA plot. It is a new axis (a combination of all 20,000 genes) that is oriented to capture the **maximum possible variation** in the dataset.
 - **Principal Component 2 (PC2):** This is the **y-axis**. It is a new axis that is perpendicular to PC1 and captures the **second-most** variation in the dataset.
- **The Goal:** To see if your samples **cluster** together in a meaningful way.

3. How to Interpret a PCA Plot

A PCA plot converts the complex correlations between samples into a simple 2D map.

- **Clustering:** Samples that are highly correlated (very similar to each other) will **cluster together** on the plot.
- **Separation:** Samples that are very different will be **far apart**.
- *Example:* In a well-run experiment, you would hope to see all your "Control" samples in one cluster and all your "Treated" samples in a *different* cluster.
- **Axis Importance:** The axes are *ranked*.
 - PC1 (the x-axis) is the most important component. Differences between clusters *horizontally* represent the *largest source of variation* in your data.
 - PC2 (the y-axis) is the second most important.

- This means two clusters separated by 5 units on the x-axis are **more different** than two clusters separated by 5 units on the y-axis.

4. The Scree Plot: Is Your PCA Plot Useful?

- A PCA plot is only a 2D *summary*. It might be leaving out a lot of important information.
- A **Scree Plot** is a bar chart that tells you **how much variation** each Principal Component (PC) explains.
- **How to Read It:**
 - The x-axis lists the PCs in order (PC1, PC2, PC3, ...).
 - The y-axis shows the "Percent of Variation Explained."
- **What You Want to See:** You want to see that PC1 and PC2 (the two axes on your plot) explain a **large percentage** of the total variation (e.g., PC1 explains 50%, PC2 explains 20%). This tells you that your 2D plot is a good and reliable summary of the data.
- **What is Bad:** If PC1 only explains 3% and PC2 only explains 2%, your 2D plot is **not useful**. It means the plot is flat and you're not capturing the true structure of the data.

This video provides a gentle overview of RNA-seq, which is one of the topics you asked about.

[StatQuest: A gentle introduction to RNA-seq](#)

http://googleusercontent.com/youtube_content/10